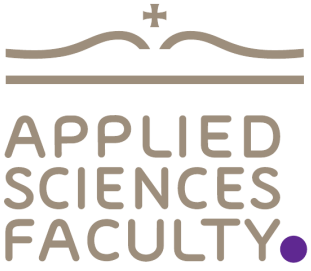
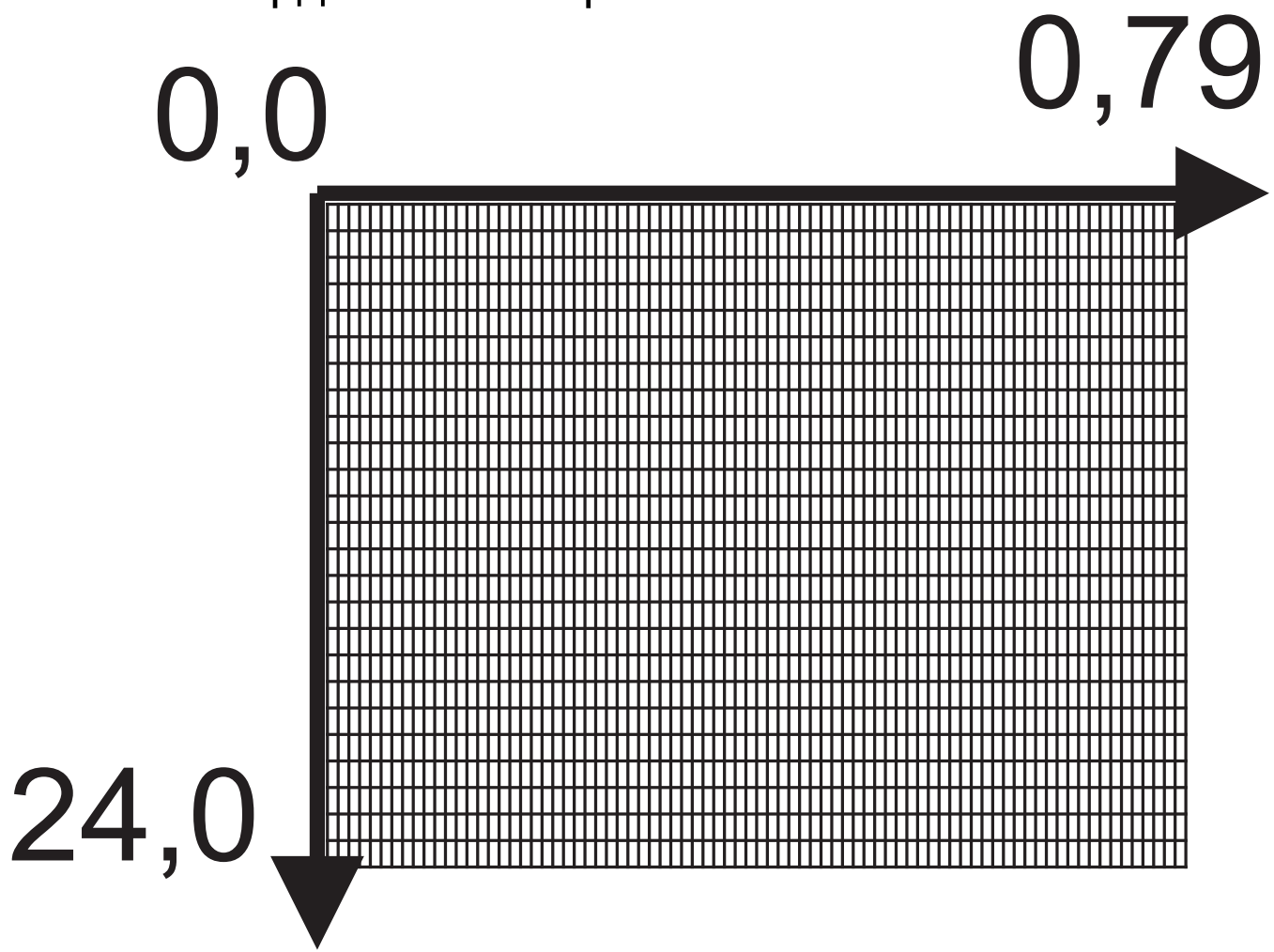
C++ — додаток. PDCurses





©Олег Фаренюк,
Creative Commons - Attribution Share Alike (CC BY-SA)
https://www.facebook.com/oleg.farenyuk
indrekis@gmail.com

Координати літер в консолі Windows:



Вказані розміри – по замовчуванню. Їх можна змінювати! Для інших операційних систем вони, тим більше, можуть відрізнятися.

PDCurses

- PDCurses — крос-платформенна бібліотека для роботи з консоллю, із "сімейства"бібліотек curses.
- Офіційна сторінка: `http://pdcurses.sourceforge.net/`.
- Скачати бінарну версію можна тут: `http://sourceforge.net/projects/pdcurses/files/pdcurses/3.4/`, файл pdc34dllw.zip.
- Офіційна сторінка іншої (сумісної) реалізації, ncurses: `http://www.gnu.org/software/ncurses/`.

Документація

- **Загальна інформація** про сімейство: `http://en.wikipedia.org/wiki/Curses_(programming_library)`. Також див. `http://en.wikipedia.org/wiki/Ncurses`.
- **Документація PDCurses**: `http://pdcurses.sourceforge.net/doc/PDCurses.txt`. Містить список типів даних, глобальних змінних та функцій бібліотеки. Описано, зокрема, відмінності від інших реалізацій.
- Офіційна сторінка іншої (в основному сумісної) реалізації, **ncurses**: `http://www.gnu.org/software/ncurses/`.
- **Підручник по ncruses**, який цілком успішно можна застосувати і до PDCurses: `http://www.tldp.org/HOWTO/NCURSES-Programming-HOWTO/`.
- **Ще один підручник**, від розробників ncruses, "Writing Programs with NCURSES": `http://invisible-island.net/ncurses/ncurses-intro.html`.
- **Довідник по ncurses**: `http://invisible-island.net/ncurses/man/ncurses.3x.html`.
- Curses Development Kit, CDK: `http://invisible-island.net/cdk/` — набір готових віджетів для curses, таких як кнопки, слайдери, списки, меню, діалоги, календарі, тощо.

Використання з проектом Visual Studio — підготовка

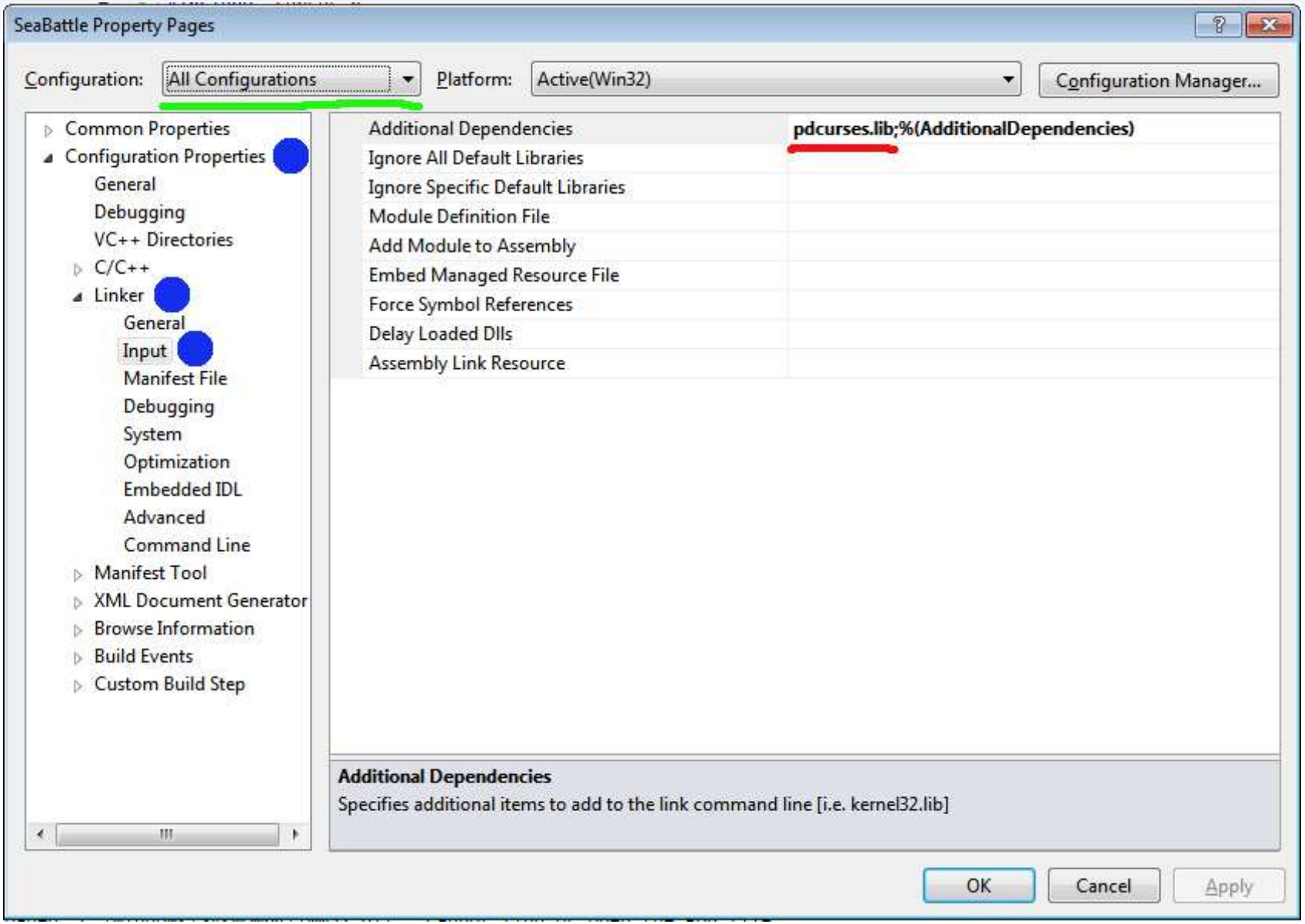
- Спершу слід скачати архів із скомпільованою бібліотекою[1]. Взяти його можна тут: `http://sourceforge.net/projects/pdcurses/files/pdcurses/3.4/`.
- Архів **pdc34dllw.zip**[2] містить наступні файли: curses.h, panel.h, pdcurses.lib, pdcurses.dll.
- **curses.h** і **panel.h** — файли заголовків, із оголошеннями всіх сутностей бібліотеки.
- **pdcurses.lib** містить інформацію для компонувальника. Це — всього лиш спеціальний архів об'єктних файлів.
- Останній із файлів, **pdcurses.dll**, є динамічною бібліотекою, яка містить, власне, реалізації, визначення, функцій. pdcurses.lib, фактично, лише "перенаправляє"компонувальник до неї.

[1] Якщо, звичайно, немає бажання компілювати самостійно.

[2] Суфікс "w"в імені означає, що бібліотеку спомпільовано з підтримкою "широких wide, символів.

Використання з проектом Visual Studio — найпростіший варіант

- Середовище повинно знати, де шукати файли бібліотеки.
- Найпростіший спосіб — покласти їх у ту ж директорію, де знаходяться .cpp-файли. Копіюємо `curses.h`, `panel.h`, `pdcurses.lib` та `pdcurses.dll` туди.
- Щоб компонувальник (linker) знав, що слід компонувати проект разом із бібліотекою, треба вказати йому використовувати pdcurses.lib. Для цього вписуємо його назву в Project → Properties → Configuration properties → Linker → Input → Additional Dependencies.
- Зауваження: щоб можна було компонувати і в Release і в Debug режимах, слід вибрати: `Configurations: All Configurations`. (Див. рисунки далі).


SeaBattle Property Pages
Configuration: All Configurations
Platform: Active(Win32)
Configuration Manager...
Common Properties
Configuration Properties
General
Debugging
VC++ Directories
C/C++
Linker
General
Input
Manifest File
Debugging
System
Optimization
Embedded IDL
Advanced
Command Line
Manifest Tool
XML Document Generator
Browse Information
Build Events
Custom Build Step
Additional Dependencies
Ignore All Default Libraries
Ignore Specific Default Libraries
Module Definition File
Add Module to Assembly
Embed Managed Resource File
Force Symbol References
Delay Loaded Dlls
Assembly Link Resource
pdcurses.lib;%(AdditionalDependencies)
Additional Dependencies
Specifies additional items to add to the link command line [i.e. kernel32.lib]
OK
Cancel
Apply

Більш професійне підключення до проекту Visual Studio – 1

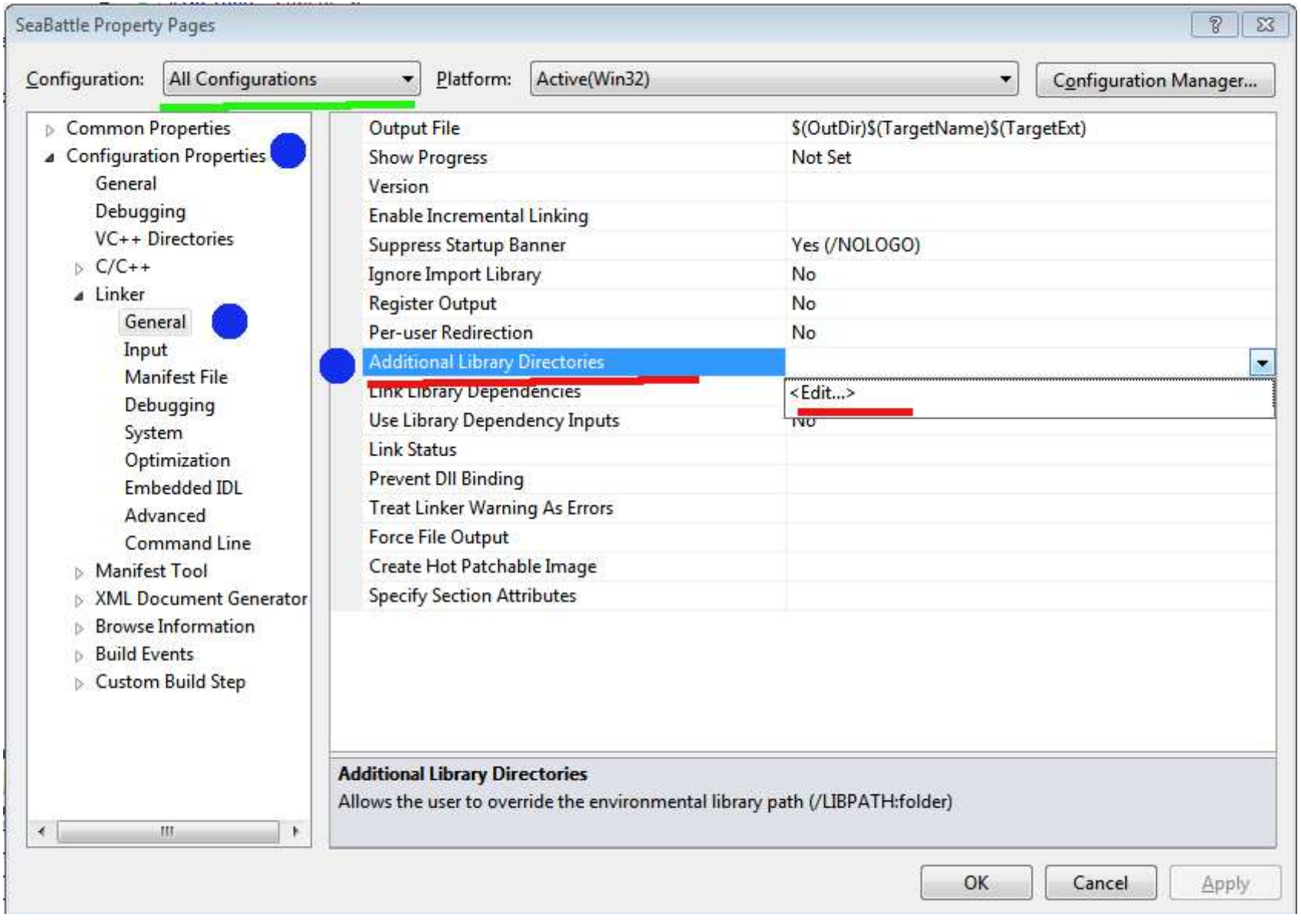
- Описаний вище спосіб простий, однак, відверто неелегантний. Крім того, якщо використовувати його, в кожному проекті буде "своя"копія файлів бібліотеки. Це непрактично, особливо для великих бібліотек. Краще зробити наступним чином:
- Файли заголовків та lib-файли зберегти у місці, спеціально виділеному для бібліотек. А IDE вказати, де їх шукати. Dll-файл покласти у директорію, яка є в змінній середовища PATH. Зокрема, це може бути щось типу `C:\Windows\System32`[3]
- Вказати шлях, де компілятор буде додатково шукати файли заголовків, *.h, можна тут: Project → Properties → Configuration properties → C/C++ → General → Additional Include Directories. Як і в найпростішому варіанті, краще це зробити для всіх конфігурацій, (`Configurations: All Configurations`).

[3]Хоча й використання цієї директорії має ряд застережень.

Більш професійне підключення до проекту Visual Studio – 2

- Шлях, де шукати додаткові бібліотеки, файли *.lib, прописуємо тут: Project → Properties → Configuration properties → Linker → General → Additional Library Directories.
- Обидва поля введення шляхів мають кнопку із трикутничком вниз, при натисканні якої з'являється випадне меню із пунктом «Edit...>". В діалозі, який з'являється, коли цей пункт вибрати, тиснемо іконку "New line"(або, на клавіатурі — Ctrl-Insert), вводимо або вибираємо шлях.
- Шляхи можна вказувати абсолютні, або відносно поточного проекту, наприклад: **$(SolutionDir)**\project_8_1\Curses.
- Зрозуміло, що файл pdcurses.lib все рівно слід вказати в Project → Properties → Configuration properties → Linker → Input → Additional Dependencies.

Ілюстрація вказання додаткових шляхів до бібліотечних файлів



Ініціалізація PDCurses

Щоб можна було скористатися функціями з цієї бібліотеки, слід включити відповідний заголовок:

```
#include "curses.h"
```

Перш ніж звертатися до неї, слід викликати функцію initscr() — своєрідний ідейний аналог конструктора для бібліотеки. Завершивши роботу, слід викликати endwin() — аналог деструктора:

```
int main() {
    initscr();  /* Ініціалізувати PDCurses*/

    КОД, ЯКИЙ ВИКОРИСТОВУЄ PDCurses

    endwin();    /* Завершити роботу з PDCurses*/
    return 0;
}
```

initscr() готує бібліотеку до роботи, виділяє пам'ять під всілякі службові структури, тощо. endwin() звільняє виділену пам'ять, переводить консоль у вихідний стан, і взагалі, "чистить"за бібліотекою.

Перелік найважливіших для нас функцій PDCurses

Бібліотека має багато різноманітних функцій. Ми використовуватимемо лише деякі із них. Детальніше — див. посилання вище.

- Виведення заданого символу у вказаних координатах:

```
int mvaddch(int y, int x, const chtype ch);
```

- Відобразити зміни (зроблені mvaddch чи іншими функціями) на екрані:

```
int refresh();
```

- Затримка, в мілісекундах:

```
int delay_output(int ms);
```

- Чекати на натискання клавіші:

```
int getch();
```

- Щоб getch() обходилася без Enter, слід викликати:

```
int raw();        або        int cbreak();
```

(У нас — ввімкнено по замовчуванню!)

- Протилежні до них

```
int noraw();        або        int nocbreak();
```

Функції, пов'язані із вводом – 1

- Для того, щоб getch() не чекав на натиснення клавіш, зупиняючи всю програму, можна використати функцію halfdelay():

```
int halfdelay(int tenths);
```

Вона змушує чекати вказану кількість десятих секунди, якщо клавіш не було натиснуто – продовжувати, повернувши код ERR:

```
halfdelay(1);
while(true){
    delay_output(500);
    printw(".");
    if(getch()!=ERR)
        break;
}
```

На жаль, halfdelay() погано дружить із іншими функціями вводу (такими як getstr(), scanw(), тощо)...

- Щоб "відмінити"дію halfdelay(), можна використати nocbreak()[4]. Після нього, якщо потрібно користуватися getch() без Enter, слід знову викликати raw() або cbreak().

[4] Документація згадує й інші засоби, але під VS 2010, із PDCurses 3.4, вони не працюють

Функції, пов'язані із вводом – 2

- Щоб заборонити echo — відображення натиснутих клавіш, можна використати:

```
int noecho();
```

- Щоб дозволити echo:

```
int echo();
```

- Для того, щоб "повернути"прочитаний символ назад, в буфер, можна використати:

```
int ungetch(int ch);
```

- Щоб очистити буфер від вже введеного:

```
int flushinp();
```

Функції, пов'язані із вводом – 3

- Функції для читання цілої стрічки[5]:

```
int getstr(char *str);
int getnstr(char *str, int n);
int mvgetstr(int y, int x, char *str);
int mvgetnstr(int y, int x, char *str, int n);
```

- getstr() читає з поточної позиції курсора до кінця рядка (або "Enter"). Вразлива до переповнення буфера!
- getnstr() поводиться так само, як getstr(), за винятком того, що читає не більше n символів. (Не забуваємо про місце для `'\0'`!)
- mvgetstr() і mvgetnstr() роблять те ж, але починаючи з вказаної позиції.

Крім того, є ціле сімейство scanw-функцій, аналогічних до стандартних scanf-функцій C. Ми їх детальніше не розглядатимемо.

[5] Не дуже дружать із halfdelay

Функції, пов'язані із вводом – 4

- Функції для читання як стрічки тесту на екрані:

```
int instr(char *str);
int innstr(char *str, int n);
int mvinstr(int y, int x, char *str);
int mvinnstr(int y, int x, char *str, int n);
```

- innstr(), mvinnstr() повертають кількість прочитаних літер, або ERR, їх варіанти без **n**, instr(), mvinstr() — OK або ERR.

Детальніше про організацію PDCurser

- Всі маніпуляції, насправді, виконуються із **вікнами**. Детальніше ми їх не розглядатимемо. Багато із розглянутих функцій мають "напарників призначених для роботи із вікнами, наприклад mv**w**addch для mvaddch.
- Насправді, функції без **w** працюють із вікном по замовчуванню, якому відповідає ціле вікно-консоль. Вказівник на це вікно знаходиться у глобальній змінній stdscr. Тобто, наступні два рядки еквівалентні:

```
mvaddch(5, 5, '*');

mvwaddch(stdscr,5, 5, '*');
```

- Є ряд функцій, яким завжди треба передавати вказівник на поточне вікно.

Курсори та розміри

- Отримати поточне розташування курсору

```
int getcury(WINDOW *win);
int getcurx(WINDOW *win);

void getyx(WINDOW *win, int y, int x);
```

  Увага! getyx() — макрос, який змінює свої аргументи y та x.

- Отримати максимальні можливі координати:

```
int getmaxy(WINDOW *win);
int getmaxy(WINDOW *win);

void getmaxyx(WINDOW *win, int y, int x);
```

  Увага! getmaxyx() — макрос, який змінює свої аргументи y та x.

- Функція переміщення курсору:

```
int move(int y, int x);
```

Функції, пов'язані із виводом – 1

- На додачу до вже описаної:

```
int mvaddch(int y, int x, const chtype ch);
```

  існують й інші функції:

```
int addch(const chtype ch);
```

  — виводить символ в положенні курсору, курсор переміщає на наступне положення.

```
int echochar(const chtype ch);
```

  — вивести символ і викликати refresh().
- Сімейство функцій для виводу стрічок:

```
int addchstr(const chtype *ch);
int addchnstr(const chtype *ch, int n);
int mvaddchstr(int y, int x, const chtype *ch);
int mvaddchnstr(int y, int x, const chtype *ch, int n);
```

  Ці функції не переміщають курсор, не переходять на новий рядок, і т.д.

Існує також сімейство функцій printw(), аналогічних до стандартних функцій C, printf. Ми їх поки не розглядатимемо.

Функції, пов'язані із виводом – 2, атрибути

- За допомогою наступних функцій:

```
int attron(chtype attrs);
int attroff(chtype attrs);
```

можна вмикати та вимикати атрибути виводу тексту.

Поміж можливих атрибутів варто згадати:

- `A_BLINK` — мигання, або світлий фон.
- `A_BOLD` — "жирний або яскравий, текст.
- `A_INVIS` — невидимий текст.
- `A_STANDOUT` — максимальне виділення, доступне терміналу.
- `A_UNDERLINE` — підкреслений, або виділений кольором, текст.

Звичайно, існують й інші.

Атрибути можна комбінувати між собою, за допомогою **бітового** OR — '|'.

Функції, пов'язані із виводом – 3, символи псевдографіки

- ACS_ULCORNER — upper left box corner
- ACS_LLCORNER — lower left box corner
- ACS_URCORNER — upper right box corner
- ACS_LRCORNER — lower right box corner
- ACS_RTEE — right "T"
- ACS_LTEE — left "T"
- ACS_BTEE — bottom "T"
- ACS_TTEE — top "T"
- ACS_HLINE — horizontal line
- ACS_VLINE — vertical line
- ACS_PLUS — plus sign, cross, or four-corner piece

Функції, пов'язані із виводом – 4, інші

- Пара допоміжних функцій:

```
int beep();
int flash();
```

— писк і мигання екрану.

- Хоча PDCurses портовано для Linux, більш "звичним"там є ncurses, тому компіляцію розглянемо на його прикладі.

```
[testcmp]$ wget -c \
 http://ftp.gnu.org/pub/gnu/ncurses/ncurses-5.9.tar.gz
..................
[testcmp]$ ls
ncurses-5.9.tar.gz
[testcmp]$ tar xvf ncurses-5.9.tar.gz
..................
[testcmp]$ pwd
/home/indrekis/testcmp
[testcmp]$ cd ncurses-5.9
[ncurses-5.9]$ ./configure \
--prefix=/home/indrekis/testcmp/installed
..................
[ncurses-5.9]$ make
..................
[ncurses-5.9]$ make install
..................
[ncurses-5.9]$ cd ..
[testcmp]$ ls installed/
bin  include  lib  man  share
```

PDCurses

- Скачуємо PDCurses-3.4 (саме цю версію – деталі від версії до версії можуть змінюватися): `http://sourceforge.net/projects/pdcurses/files/pdcurses/3.4/PDCurses-3.4.tar.gz/download`.
- Розархівовуємо, переходимо у відповідну директорію.
- Переходимо у її піддиректорію win32
- У файлі mingwin32.mak замінюємо в шляхах слеші "\"на "/":

```
BASEDEF  = $(PDCURSES_SRCDIR)/exp-base.def
WIDEDEF  = $(PDCURSES_SRCDIR)/exp-wide.def
```

  В ньому ж, замінити type на cat (двічі), copy на cp (Один раз), del на rm (тричі).
- Виконуємо:

```
make -f mingwin32.mak  DLL=Y WIDE=Y UTF8=Y all
```

- Отримані файли (відносно кореневої директорії джерельних текстів): win32/panel.a, win32/pdcurses.a, win32/pdcurses.dll, ./curses.h, ./curspriv.h, ./panel.h, ./term.h.











